Annexe : Contribution de l'IA au développement du prototype

Architecture Fullstack Minimaliste pour Noyau Python

Généré par Assistant IA

2026-11-11

Table of contents

1 Introduction et Objectifs **1**

2 Architecture Logique Proposée **2**

3 Code Source du Prototype **2**
3.1 Structure du Projet 2
3.2 Le Serveur Backend (`main.py`) 2
3.3 Le Client Frontend (`templates/index.html`) 3

4 Résolution d'Incidents Techniques **6**
4.1 Correction du Bug `TypeError: unhashable type: 'dict'` 6
4.2 Exécution sous environnement Windows 6

1 Introduction et Objectifs

Ce document consigne les échanges et les contributions de l'intelligence artificielle dans la conception du squelette d'application web *fullstack*. L'objectif principal était de concevoir une interface client-serveur ultra-légère afin de minimiser le temps de mise en route (*ramp-up*) et de concentrer l'effort de développement sur le noyau fonctionnel en Python.

2 Architecture Logique Proposée

L'application repose sur deux composants principaux sans framework frontal lourd : 1. **Serveur (Backend) :** FastAPI (Python) pour une intégration native et rapide avec le noyau. 2. **Client (Frontend) :** HTML5, CSS3 et JavaScript natif (Vanilla JS) structuré en deux volets (paramètres à gauche, résultats dynamiques à droite).

3 Code Source du Prototype

3.1 Structure du Projet

```
mon_projet/
   main.py          # Le serveur FastAPI
   templates/
       index.html   # La page web (HTML/CSS/JS)
```

3.2 Le Serveur Backend (`main.py`)

```
from fastapi import FastAPI, Request, Query
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from datetime import date
from typing import Optional

app = FastAPI()
templates = Jinja2Templates(directory="templates")

# Route pour afficher la page d'accueil
@app.get("/", response_class=HTMLResponse)
async def read_item(request: Request):
    return templates.TemplateResponse("index.html", context={"request": request})

# Route API en GET qui reçoit les paramètres dans l'URL
@app.get("/api/process")
async def process_data(
    start_date: date = Query(...),
    duration_days: int = Query(...),
    quantity: int = Query(...),
```

```
    enum_option: str = Query(...),
    flag_feat1: bool = Query(False),
    flag_feat2: bool = Query(False)
):
    # --- Intégrez votre noyau Python ici ---
    # Exemple de traitement bidon basé sur vos paramètres
    result_data = {
        "Statistiques globales": [
            f"Date de début prise en compte : {start_date}",
            f"Calcul sur une durée de {duration_days} jours.",
            f"Quantité cible : {quantity} unités."
        ],
        "Configuration appliquée": [
            f"Mode d'énumération sélectionné : {enum_option}",
            f"Option Option A active : {flag_feat1}",
            f"Option Option B active : {flag_feat2}"
        ],
        "Métriques générées par le noyau": [
            f"Résultat simulé X : {quantity * duration_days}",
            "Statut du traitement : Succès"
        ]
    }

    # Renvoie la réponse proprement en JSON
    return result_data
```

3.3 Le Client Frontend (`templates/index.html`)

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Interface Noyau Python</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; background-color: #f4f4f9; }
        .container { display: flex; gap: 40px; }
        .left-panel { flex: 1; background: white; padding: 20px; border-radius: 8px; box-shad
        .right-panel { flex: 1; background: white; padding: 20px; border-radius: 8px; box-sha
        .form-group { margin-bottom: 15px; }
        label { display: block; margin-bottom: 5px; font-weight: bold; }
```

```
        input[type="date"], input[type="number"], select { width: 100%; padding: 8px; box-siz
        .actions { margin-top: 20px; display: flex; gap: 10px; }
        button { padding: 10px 15px; border: none; border-radius: 4px; cursor: pointer; font-
        button[type="submit"] { background-color: #007bff; color: white; }
        button[type="reset"] { background-color: #6c757d; color: white; }
        h4 { color: #333; margin-top: 20px; border-bottom: 2px solid #eeeff1; padding-bottom
    </style>
</head>
<body>
    <h1>Tableau de bord du noyau de traitement</h1>
    <div class="container">
        <div class="left-panel">
            <form id="paramForm" method="GET" action="/api/process">
                <div class="form-group">
                    <label for="start_date">Date de départ :</label>
                    <input type="date" id="start_date" name="start_date" required>
                </div>
                <div class="form-group">
                    <label for="duration_days">Durée :</label>
                    <select id="duration_days" name="duration_days">
                        <option value="7">1 Semaine (7 jours)</option>
                        <option value="28">1 Mois (28 jours)</option>
                        <option value="90">1 Trimestre (90 jours)</option>
                        <option value="365" selected>1 Année (365 jours)</option>
                        <option value="3653">1 Décennie (3653 jours)</option>
                    </select>
                </div>
                <div class="form-group">
                    <label for="quantity">Quantité désirée :</label>
                    <input type="number" id="quantity" name="quantity" value="10" min="1" rec
                </div>
                <div class="form-group">
                    <label>Mode d'exécution :</label>
                    <input type="radio" id="mode1" name="enum_option" value="standard" checke
                    <input type="radio" id="mode2" name="enum_option" value="avance"> Avancé
                </div>
                <div class="form-group">
                    <label>Options (Flags) :</label>
                    <input type="checkbox" id="flag_feat1" name="flag_feat1" value="true" che
                    <input type="checkbox" id="flag_feat2" name="flag_feat2" value="true"> Op
                </div>
                <div class="actions">
```

```
                <button type="submit">Soumettre</button>
                <button type="reset" id="resetBtn">Réinitialiser</button>
            </div>
        </form>
    </div>
    <div class="right-panel" id="resultPanel">
        <p>En attente de la soumission des paramètres...</p>
    </div>
</div>

<script>
    document.getElementById('start_date').valueAsDate = new Date();
    const form = document.getElementById('paramForm');
    const resultPanel = document.getElementById('resultPanel');

    form.addEventListener('submit', async (e) => {
        e.preventDefault();
        const formData = new FormData(form);
        const params = new URLSearchParams();
        params.append('start_date', formData.get('start_date'));
        params.append('duration_days', formData.get('duration_days'));
        params.append('quantity', formData.get('quantity'));
        params.append('enum_option', formData.get('enum_option'));
        params.append('flag_feat1', document.getElementById('flag_feat1').checked);
        params.append('flag_feat2', document.getElementById('flag_feat2').checked);

        const apiUrl = `/api/process?${params.toString()}`;
        window.history.pushState({}, '', `?${params.toString()}`);

        try {
            const response = await fetch(apiUrl);
            const jsonResult = await response.json();
            resultPanel.innerHTML = "<h2>Résultats du traitement</h2>";

            for (const [sectionTitle, items] of Object.entries(jsonResult)) {
                const h4 = document.createElement('h4');
                h4.textContent = sectionTitle;
                resultPanel.appendChild(h4);
                const ul = document.createElement('ul');
                items.forEach(item => {
                    const li = document.createElement('li');
                    li.textContent = item;
```

```
                        ul.appendChild(li);
                    });
                    resultPanel.appendChild(ul);
                }
            } catch (error) {
                resultPanel.innerHTML = `<p style="color:red;">Erreur: ${error.message}</p>`
            }
        });
    </script>
</body>
</html>
```

4 Résolution d'Incidents Techniques

4.1 Correction du Bug `TypeError: unhashable type: 'dict'`

Lors du premier lancement de l'application, l'appel de rendu de gabarit suivant a levé une exception interne dans le framework de routage :

```
# CODE ERRONÉ (Syntaxe obsolète)
return templates.TemplateResponse("index.html", {"request": request})
```

Diagnostic : Les récentes versions de `starlette` / `jinja2` requièrent l'utilisation explicite du paramètre nommé `context` pour encapsuler le dictionnaire d'arguments ou l'attribution directe de `request`.

Correction appliquée :

```
# CODE CONFIGURÉ ET SÉCURISÉ
return templates.TemplateResponse("index.html", context={"request": request})
```

4.2 Exécution sous environnement Windows

Dans le cas où la variable globale d'environnement pour l'exécutable `uvicorn` est manquante (erreur d'identification de commande interne), le protocole d'exécution alternatif via le gestionnaire de modules natif de Python doit être utilisé :

```
python -m uvicorn main:app --reload
```